```
HashMap() {
  this.loadFactor = DEFAULT_LOAD_FACTOR; //0.75f
}
```

```
HashMap(int initialCapacity) { //指定初始容量
  this(initialCapacity, DEFAULT_LOAD_FACTOR);
}
```

```
HashMap(int initialCapacity, float loadFactor) { //指定初始容量和扩容因子
  if (initialCapacity < 0) throw new IllegalArgumentException("Illegal initial capacity: " + initialCapacity);
  if (initialCapacity > MAXIMUM_CAPACITY) initialCapacity = MAXIMUM_CAPACITY; //指定容量不能超过最大值 (2的30次方, 约10亿)
  if (loadFactor ≤ 0 || Float.isNaN(loadFactor)) throw new IllegalArgumentException("Illegal load factor: " + loadFactor);
  this.loadFactor = loadFactor;
  this.threshold = tableSizeFor(initialCapacity); //阈值
}
```

```
int tableSizeFor(int cap) { //返回「最接近」「不小于」cap的「2的n次方」
  int n = cap - 1;
  n | = n >>> 1;
  n | = n >>> 2;
  n | = n >>> 4;
  n | = n >>> 8;
  n | = n >>> 16;
  return (n < 0) ? 1 : (n ≥ MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n + 1;
}
```

```
HashMap(Map<? extends K, ? extends V> m) { //根据已有map创建
  this.loadFactor = DEFAULT_LOAD_FACTOR;
  putMapEntries(m, false);
}
```

```
void putMapEntries(Map<? extends K, ? extends V> m, boolean evict) {
  int s = m.size();
  if (s > 0) {
    //1.根据m.size, 调整map容量
    if (table == null) { //1.1 map未初始化
      float ft = ((float)s / loadFactor) + 1.0F;
      int t = ((ft < (float)MAXIMUM_CAPACITY) ? (int)ft : MAXIMUM_CAPACITY);
      if (t > threshold) threshold = tableSizeFor(t);
    } else if (s > threshold) resize(); //1.2 容量超过阈值, 扩容
    //2.遍历m, 将每个元素copy到当前map
    for (Map.Entry<? extends K, ? extends V> e : m.entrySet()) {
      K key = e.getKey();
      V value = e.getValue();
      putVal(hash(key), key, value, false, evict);
    }
  }
}
```

```
int hash(Object key) {
  int h;
  return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
  /* 有些数据的hashCode差异性主要在高位, 但bucket的寻址算法从低位开始取,
     将hashCode高16位右移到低位, 再做异或运算, 能充分利用高位差异性,
     计算出的bucket地址分布均匀, 减少hash冲突 spreads the impact of
     higher bits downward*/
}
```

```
V putVal(int hash, K key, V value, boolean onlyIfAbsent, boolean evict) {
  Node<K,V>[] tab; Node<K,V> p; int n, i;
  if ((tab = table) == null || (n = tab.length) == 0) n = (tab = resize()).length; //1.如果table为空, 先resize() lazy-load
  if ((p = tab[i = (n - 1) & hash]) == null) tab[i] = newNode(hash, key, value, null); //2.hash不冲突, 直接存 (n-1)&hash截取了hash值的后几位
  else { //3.hash冲突
    Node<K,V> e; K k;
    if (p.hash == hash && ((k = p.key) == key || (key ≠ null && key.equals(k)))) e = p; //发现相同节点 (hash和key完全相同) , 跳转至CASE3
    else if (p instanceof TreeNode) e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value); //CASE2:红黑树
    else {
      for (int binCount = 0; ; ++binCount) { //CASE1: 加入链表
        if ((e = p.next) == null) { //找到链表尾节点
          p.next = newNode(hash, key, value, null); //加入链表尾部
          if (binCount ≥ TREEIFY_THRESHOLD - 1) treeifyBin(tab, hash); //单链表该转红黑树了
          break; //跳出循环
        }
        if (e.hash == hash && ((k = e.key) == key || (key ≠ null && key.equals(k)))) break; //发现相同节点 (hash和key完全相同) , 跳转至CASE3
        p = e; //继续寻找尾节点
      }
    }
    if (e ≠ null) { //CASE3: 存在相同节点
      V oldValue = e.value;
      if (!onlyIfAbsent || oldValue == null) e.value = value; //替换旧值
      afterNodeAccess(e); //do nothing
      return oldValue; //返回旧值
    }
  }
  ++modCount; //4.更新map的修改次数
  if (++size > threshold) resize(); //5.map.size加1, 如果超过了扩容阈值, 扩容
  afterNodeInsertion(evict); //do nothing
  return null; //6.返回null
}
```

```
Set<Map.Entry<K,V>> entrySet() {
  Set<Map.Entry<K,V>> es;
  return (es = entrySet) == null ? (entrySet = new EntrySet()) : es;
}
```

```
foreach编译后, 会转为以下代码执行:

Iterator var = entrySet.iterator();
while(var.hasNext()) {
  Entry<String, String> entry = (Entry)var.next();
  { 原foreach循环体里的逻辑 }
}
```

```
Iterator<Map.Entry<K,V>> EntrySet.iterator() {
  return new EntryIterator();
}
```

```
HashIterator() {
  expectedModCount = modCount; //记录此时map状态, 后续遍历map时核对, 如果发生变化, 说明其他线程修改了map
  Node<K,V>[] t = table;
  current = next = null; //current记录当前返回节点, next指向下次返回的节点
  index = 0; //entry在table里的坐标
  if (t ≠ null && size > 0) {
    do {} while (index < t.length && (next = t[index++]) == null); // advance to first entry
  }
  //此时, current = null; next = table[0]; index = 1
}
```

```
/* 按顺序依次返回map里的节点, 顺序为:
   table[0]里的所有节点 (按存储顺序返回)
   table[1]里的所有节点 (按存储顺序返回)
   table[2]里的所有节点 (按存储顺序返回)  ... */
Map.Entry<K,V> EntryIterator.next() {
 return nextNode();
}
```

```
HashIterator.nextNode() {
  Node<K,V>[] t;
  Node<K,V> e = next;
  if (modCount ≠ expectedModCount) throw new ConcurrentModificationException(); //其他线程改动了map (容量或结构) , fail fast
  if (e == null) throw new NoSuchElementException();
  if ((next = (current = e).next) == null && (t = table) ≠ null) { //当前entry下的链表遍历完了
    do {} while (index < t.length && (next = t[index++]) == null); // advance to next entry
  }
  return e;
}
```

```
Node<K,V>[] resize() {
  Node<K,V>[] oldTab = table;
  int oldCap = (oldTab == null) ? 0 : oldTab.length;
  int oldThr = threshold;
  int newCap, newThr = 0;
  //1.计算新容量
  if (oldCap > 0) { //1.1.原map不为空
    if (oldCap ≥ MAXIMUM_CAPACITY) { //当前容量超过最大容量
      threshold = Integer.MAX_VALUE;
      return oldTab; //无法扩容, 返回原table
    } else if ((newCap = oldCap << 1) < MAXIMUM_CAPACITY && oldCap ≥ DEFAULT_INITIAL_CAPACITY) newThr = oldThr << 1; //如果符合条件, 新容量=2*原容量
  } else if (oldThr > 0) newCap = oldThr; //1.2. 首次初始化, 有指定值, 按指定值
    else { //1.3.没指定值使用默认值
    newCap = DEFAULT_INITIAL_CAPACITY;
    newThr = (int)(DEFAULT_LOAD_FACTOR * DEFAULT_INITIAL_CAPACITY);
  }
  if (newThr == 0) {
    float ft = (float)newCap * loadFactor;
    newThr = (newCap < MAXIMUM_CAPACITY && ft < (float)MAXIMUM_CAPACITY ? (int)ft : Integer.MAX_VALUE);
  }
  //2.新建table
  threshold = newThr;
  Node<K,V>[] newTab = (Node<K,V>[])new Node[newCap];
  table = newTab; //并发问题, 如果此时其他线程访问table, 会找不到数据
  if (oldTab ≠ null) { //原table不为空, 数据迁移到新table
    for (int j = 0; j < oldCap; ++j) { //遍历bucket
      Node<K,V> e;
      if ((e = oldTab[j]) ≠ null) {
        oldTab[j] = null; //取出bucket, 存到e
        if (e.next == null) newTab[e.hash & (newCap - 1)] = e; //CASE1:单节点链表, 直接存到newTab
        else if (e instanceof TreeNode) ((TreeNode<K,V>)e).split(this, newTab, j, oldCap); //CASE2:红黑树
        else { //CASE3:单链表
          Node<K,V> loHead = null, loTail = null;
          Node<K,V> hiHead = null, hiTail = null;
          Node<K,V> next;
          do { //遍历链表的所有节点
            next = e.next;
            if ((e.hash & oldCap) == 0) { //满足低位区条件的节点组成新的链表
              if (loTail == null) loHead = e;
              else loTail.next = e;
              loTail = e;
            } else { //满足高位区条件的节点组成新的链表
              if (hiTail == null) hiHead = e;
              else hiTail.next = e;
              hiTail = e;
            }
          } while ((e = next) ≠ null);
          if (loTail ≠ null) { loTail.next = null; newTab[j] = loHead; } //低位区新链表存到newTab[j]
          if (hiTail ≠ null) { hiTail.next = null; newTab[j + oldCap] = hiHead; } //高位区新链表存到newTab[j+oldCap]
        }
      }
    }
  }
  return newTab; //3.返回新table
}
```

```
V put(K key, V value) {
  return putVal(hash(key), key, value, false, true);
}
```

```
V get(Object key) {
  Node<K,V> e;
  return (e = getNode(hash(key), key)) == null ? null : e.value;
}
```

```
Node<K, V> getNode(int hash, Object key) {
  Node<K,V>[] tab; Node<K,V> first, e; int n; K k;
  if ((tab = table) ≠ null && (n = tab.length) > 0 && (first = tab[(n - 1) & hash]) ≠ null) { //1.定位bucket
    if (first.hash == hash && ((k = first.key) == key || (key ≠ null && key.equals(k)))) return first; //2.如果第一个节点正好是, 直接返回
    if ((e = first.next) ≠ null) { //3.在bucket里找, 找到了直接返回
      if (first instanceof TreeNode) return ((TreeNode<K,V>)first).getTreeNode(hash, key); //3.1.在红黑树里找
      do { //3.2.遍历单链表找
        if (e.hash == hash && ((k = e.key) == key || (key ≠ null && key.equals(k)))) return e;
      } while ((e = e.next) ≠ null);
    }
  }
  return null; //4.遍历完也没找到, 返回空
}
```

```
bucket坐标规律: 扩容后的坐标 = 上次坐标 + oldCap * (1 or 0) (取1还是0由截取的hash值最高位决定)
数组容量n为16时:                 index0 = 15 & hash,   相当于截取hash二进制的后4位
数组容量n为32时 (第1次扩容) :   index1 = 31 & hash,   相当于截取hash二进制的后5位 (index0 + 第5位*16)
数组容量n为64时 (第2次扩容) :   index2 = 63 & hash,   相当于截取hash二进制的后6位 (index1 + 第6位*32)
数组容量n为128时 (第3次扩容) : index3 = 127 & hash, 相当于截取hash二进制的后7位 (index2 + 第7位*64)
数组容量n为256时 (第4次扩容) : index4 = 255 & hash, 相当于截取hash二进制的后8位 (index3 + 第8位*128)
```

```
void putAll(Map<? extends K, ? extends V> m) {
  putMapEntries(m, true);
}
```

```
V remove(Object key) {
  Node<K,V> e;
  return (e = removeNode(hash(key), key, null, false, true)) == null ? null : e.value;
}
```

```
Node<K, V> removeNode(int hash, Object key, Object value,boolean matchValue, boolean movable) {
  Node<K,V>[] tab; Node<K,V> p; int n, index;
  if ((tab = table) ≠ null && (n = tab.length) > 0 && (p = tab[index = (n - 1) & hash]) ≠ null) { //1.定位bucket
    Node<K,V> node = null, e; K k; V v;
    //2.在bucket里寻找要删除的node
    if (p.hash == hash && ((k = p.key) == key || (key ≠ null && key.equals(k)))) node = p; //恰好是第1个node, 标记下来
    else if ((e = p.next) ≠ null) {
      if (p instanceof TreeNode) node = ((TreeNode<K,V>)p).getTreeNode(hash, key); //在红黑树里寻找
      else {
        do { //在单链表里寻找
          if (e.hash == hash && ((k = e.key) == key || (key ≠ null && key.equals(k)))) { node = e; break; }
          p = e;
        } while ((e = e.next) ≠ null);
      }
    }
    //3.如果找到了, 执行删除
    if (node ≠ null && (!matchValue || (v = node.value) == value || (value ≠ null && value.equals(v)))) {
      if (node instanceof TreeNode) ((TreeNode<K,V>)node).removeTreeNode(this, tab, movable); //在红黑树里删
      else if (node == p) tab[index] = node.next; //如果是第1个node, 下一个node上位
      else p.next = node.next; //在单链表里删
      ++modCount; //更新map统计数据
      --size;
      afterNodeRemoval(node);
      return node; //返回原node
    }
  }
  return null; //4.如果没找到, 返回null
}
```